AVL Trees

[CSE2010] Data Structures

Department of Data Science

Balanced Search Trees

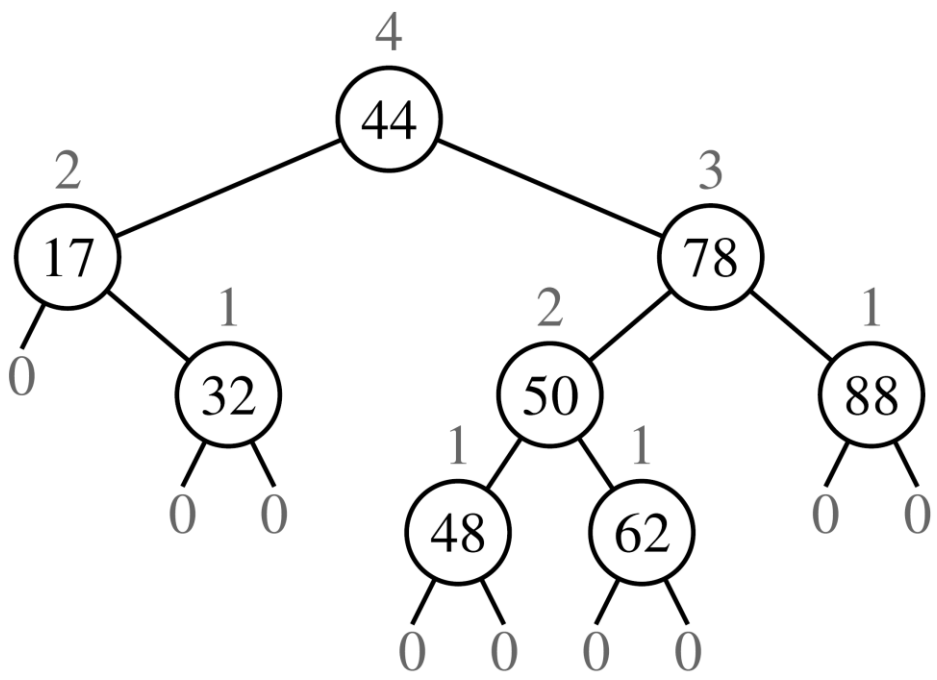
- Assume a random series of **insertions** and **removals** into a binary search tree.
- Standard binary search tree supports
  - **O(logn) expected running time** for search.
  - unfortunately, **O(n) worst-case time**
    - some sequences of operations may lead to an unbalanced tree with height proportional to **n**.
- Is it possible to augment a binary search tree to provide stronger performance guarantees?

Adelson-Velskii-Landis (AVL) Trees

Binary search tree where for every node in the tree, the heights of its left subtree and right subtree differ by at most 1.

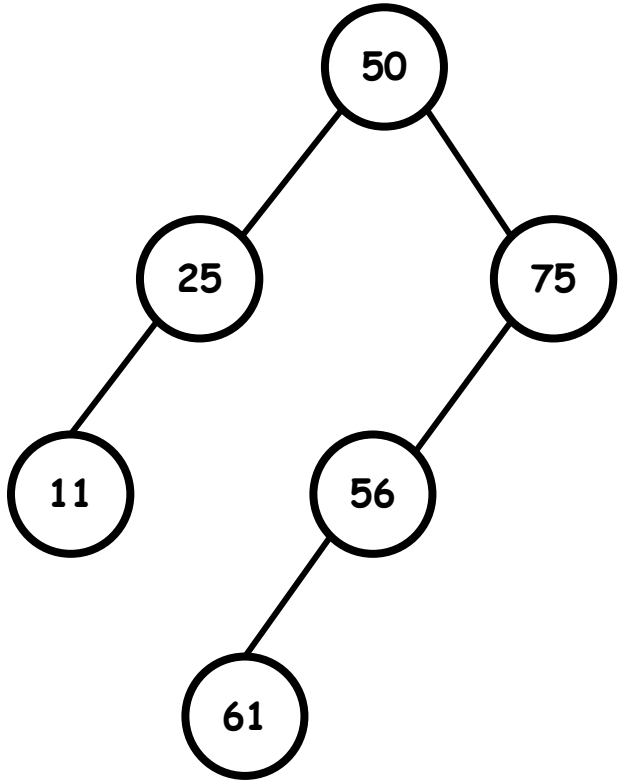
AVL Tree

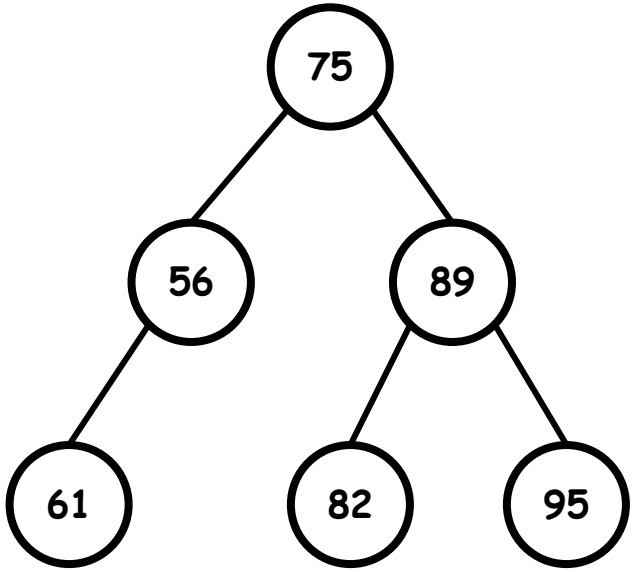
- Binary tree is said to be an AVL tree if
  1. **Binary search tree**
  2. For each node, the height of the left subtree and the height of the right subtree differ by at most 1 (**balanced**).



AVL Trees

- Are they AVL trees?





Why does an AVL tree try to keep the heights of the left and right subtrees balanced for every node?

Height of AVL Tree

- **Proposition: The height of an AVL tree with $\boldsymbol{n}$ nodes is $\boldsymbol{O(logn)}$.**
- **Justification:** Find a lower bound on the minimum number of nodes n_h of an AVL tree with height h.
- Base case: $n_1 = 1$, $n_2 = 2$

Height of AVL Tree

- **Proposition: The height of an AVL tree with n nodes is $O(logn)$.**
- **Justification:** Find a lower bound on the minimum number of nodes n_h of an AVL tree with height h.
- Recurrence for $h \geq 3$: $n_h = 1 + n_{h-1} + n_{h-2}$ (1)

  $n_h = 1 + n_{h-1} + n_{h-2} > 2 \cdot n_{h-2} > 4 \cdot n_{h-4} > 8 \cdot n_{h-6} > 2^i \cdot n_{h-2i}$ (2)
- Pick i so that $h - 2i = 1$ or $h - 2i = 2$
  - $i = \left\lceil \frac{h}{2} \right\rceil - 1$
- $n_h > 2^i \cdot n_{h-2i} = 2^{\left\lceil \frac{h}{2} \right\rceil - 1} \cdot n_{h-2\left\lceil \frac{h}{2} \right\rceil + 2} \geq 2^{\left\lceil \frac{h}{2} \right\rceil - 1} \cdot n_1 \geq 2^{\frac{h}{2} - 1}$ (3)
- $\log(n_h) > \frac{h}{2} - 1$ (4)
- $h < 2\log(n_h) + 2 \Leftrightarrow h = O(logn)$ (5)

Implication of O(logn) Height

- The **height** of an AVL tree with n nodes is $\boldsymbol{O(logn)}$.

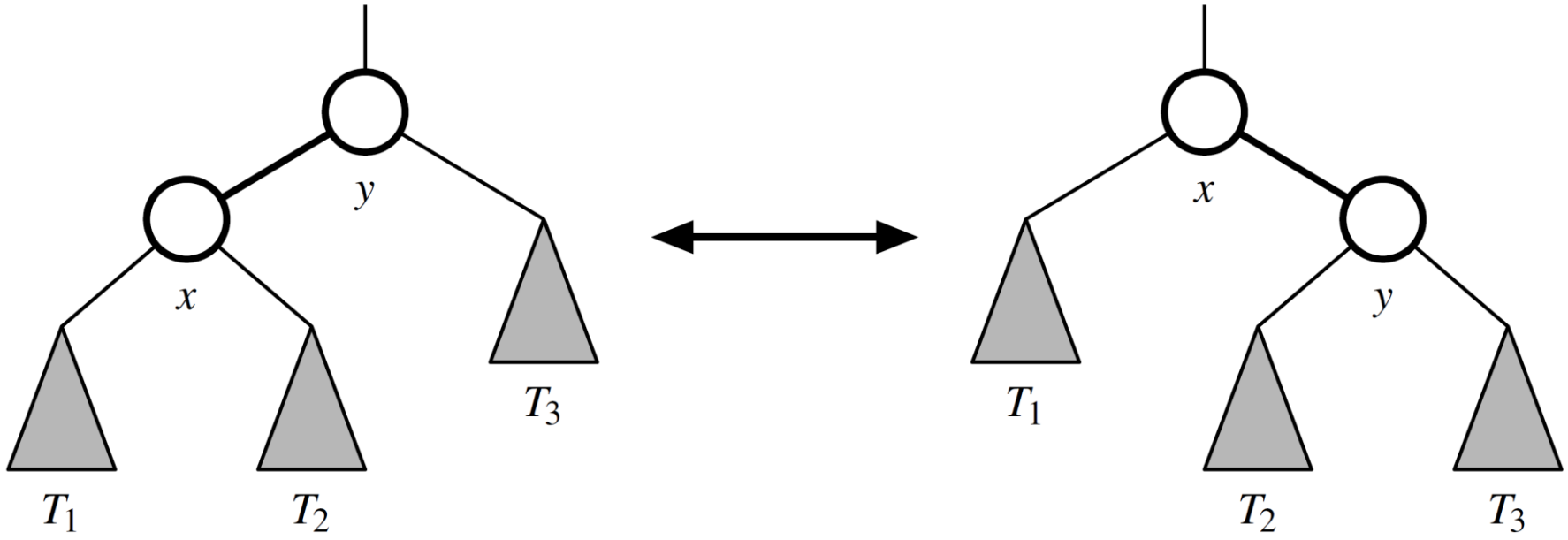
$$\Downarrow$$

- Main operations (search, insertion, deletion) in an AVL tree take $\boldsymbol{O(logn)}$ time.

How does an AVL tree try to keep the **heights** of the left and right subtrees **balanced** for every node?

Rotation

- Occasionally reshape the tree and reduce its height.
- Primary operation to rebalance a binary search tree is **rotation.**



Trinode Restructuring

- Given a node x, its parent y, and its grandparent z
- **Goal**: restructure the subtree rooted at z in order to reduce the overall path length to x and its subtrees.

Restructure(x):

Input: node x, x's parent y, x's grandparent z

Output: Tree T after a trinode restructuring involving x, y, and z

$(a, b, c) \leftarrow$ inorder listing of x, y, z

$(T_1, T_2, T_3, T_4) \leftarrow$ inorder listing of four subtrees of x, y, and z (not rooted at x, y, or z).

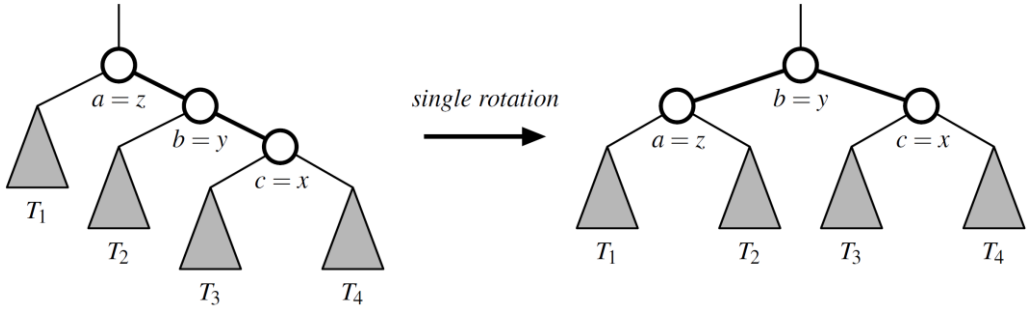
Replace the subtree rooted at z with a new subtree rooted at b.

$a \leftarrow$ left child of b

$(T_1, T_2) \leftarrow$ left and right subtrees of a, respectively.

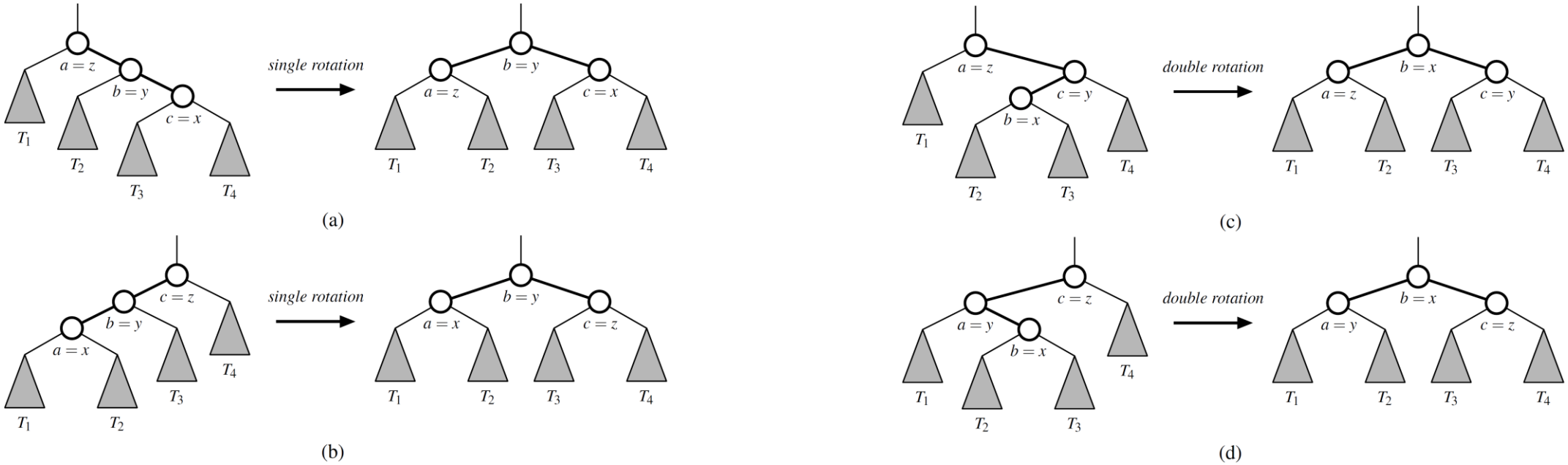
$c \leftarrow$ right child of b

$(T_3, T_4) \leftarrow$ left and right subtrees of c, respectively.



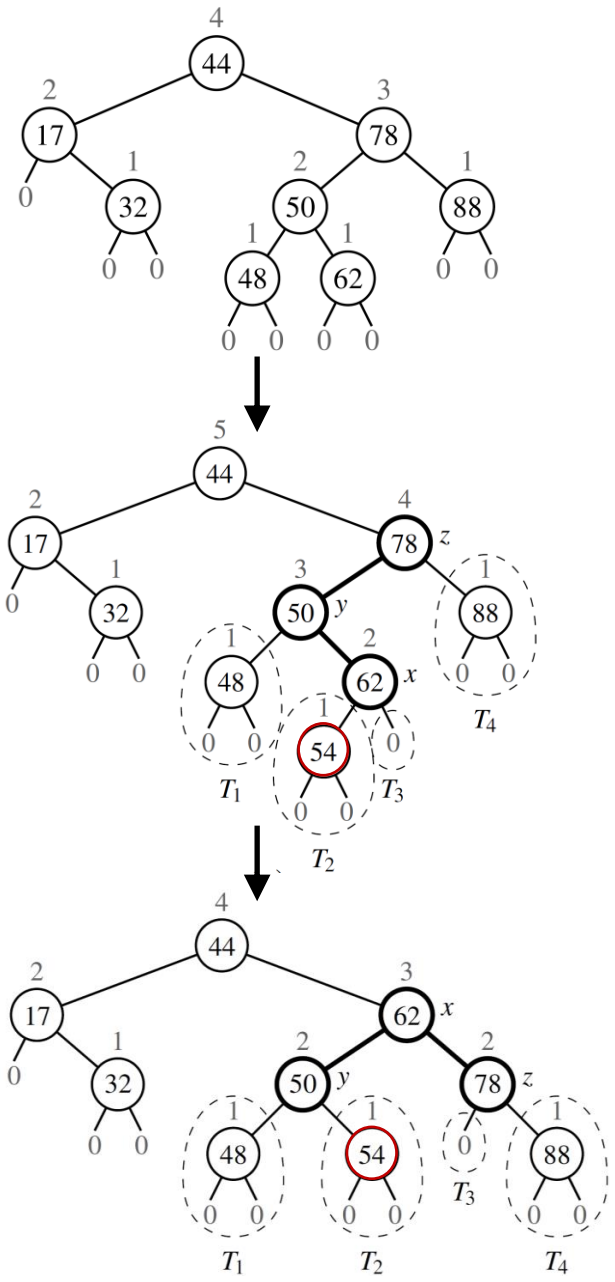
Trinode Restructuring

- Given a node x, its parent y, and its grandparent z:
- **Goal**: restructure the subtree rooted at z in order to reduce the overall path length to x and its subtrees.



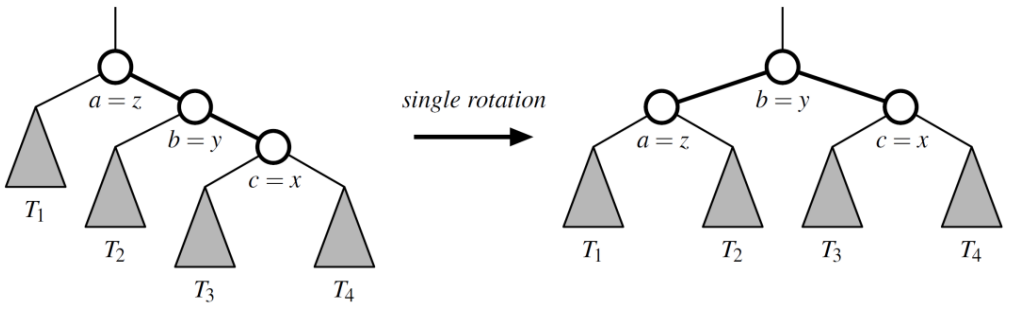
Insertion

- Suppose that we insert a new node at a leaf p.
  - The only nodes that may become unbalanced are **ancestors of p**.
- $\boldsymbol{z}$: the nearest ancestor of p that became unbalanced after the insertion of p
- $\boldsymbol{y}$: the child of z with higher height
- $\boldsymbol{x}$: the child of y with higher height
  - (possibly p itself)
- Rebalance the subtree rooted at z by calling the **trinode restructuring** method, $\mathbf{Restructure}(\boldsymbol{x})$



Insertion

1. Perform the BST **insertion** of a new node w
2. From w, travel up and find the first unbalanced node.
   - z: the first unbalanced node,
   - y: the child of z that comes on the path from w to z,
   - x: the grandchild of z that comes on the path from w to z.
3. Rebalance the tree by performing appropriate rotations on the subtree rooted at z.
   - Nodes x, y, z are handled in one of 4 cases.



```
Insert(root, key) // Here, root is the root of the current subtree
  if root = NULL
    root ← Create a new node
    root.key ← key
    root.height ← 0
    root.left ← root.right ← NULL
  else if key < root.key
    root.left ← Insert(root.left, key)
    if root.left.height – root.right.height > 1
      if key < root.left.key
        root ← LeftLeft(root)
      else
        root ← LeftRight(root)
  else if key > root.key
    root.right ← Insert(root.right, key)
    if root.right.height – root.left.height > 1
      if key < root.right.key
        root ← RightLeft(root)
      else
        root ← RightRight(root)
  root.height ← max(height(root.left), height(root.right)) + 1
  return root
```

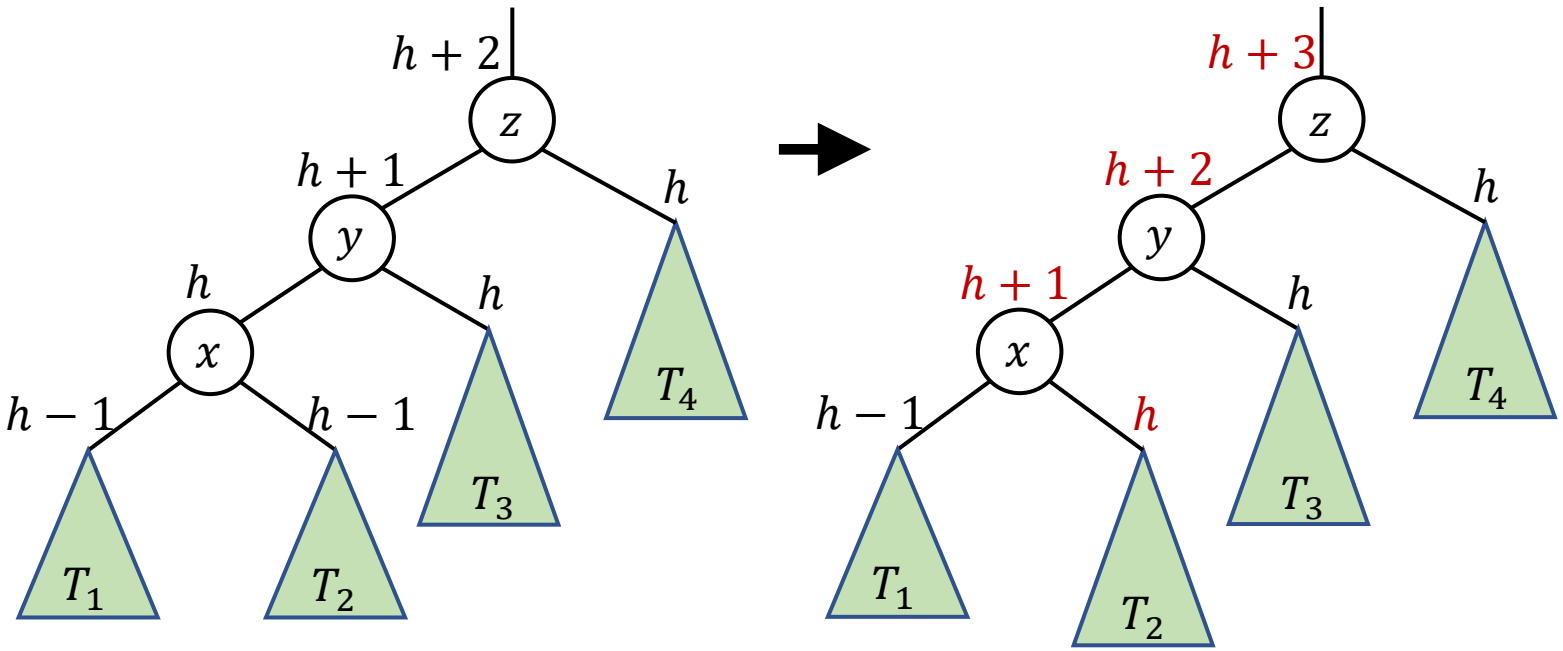
- Variable **root** is the root of the current subtree.
- When a current subtree has no node, insert a new node here
- When a new key is smaller than the root's key,
  (1) try to insert the key into the left subtree,
  (2) balance the root if it becomes not balanced
- When a new key is larger than the root's key,
  (1) try to insert the key into the right subtree,
  (2) balance the root if it becomes unbalanced
- Update the height of the root
  Here, height(x) = -1 if x is NULL; x.height otherwise

Insertion

- Unbalanced node "z" ($|h_L - h_R| > 1$) needs to be rebalanced, when
  - Case 1 (LL): insertion into the left subtree of the left child of node z
    → single rotation
  - Case 2 (RR): insertion into the right subtree of the right child of node z
    → single rotation
  - Case 3 (LR): insertion into the right subtree of the left child of node z
    → double rotation
  - Case 4 (RL): insertion into the left subtree of the right child of node z
    → double rotation

Case 1 (LL Rotation)

- Insertion into **the left subtree of the left child** of node z



Case 1 (LL Rotation)

LeftLeft(z):

y ← z.left

z.left ← y.right

y.right ← z

z.height ← max(height(z.left), height(z.right)) + 1

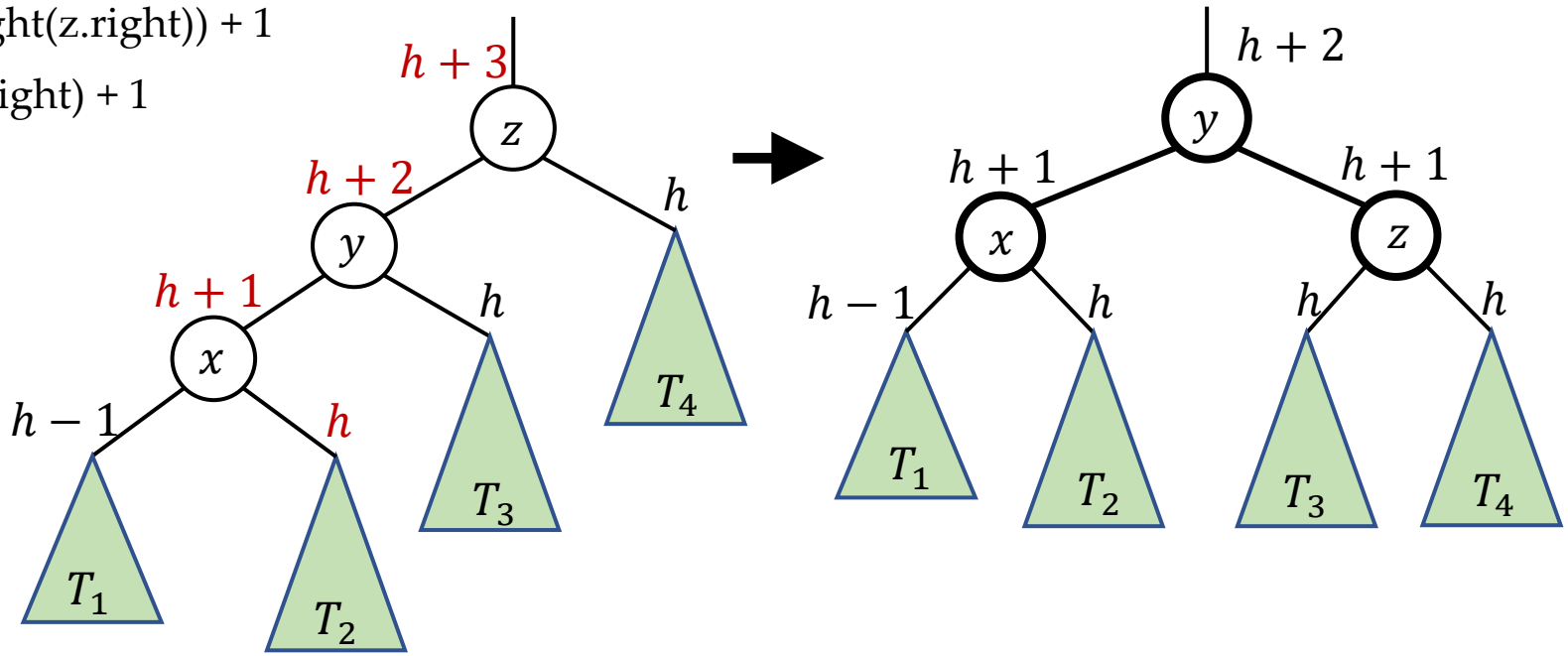
y.height ← max(height(y.left), z.height) + 1

return y /* new root */

height(z):

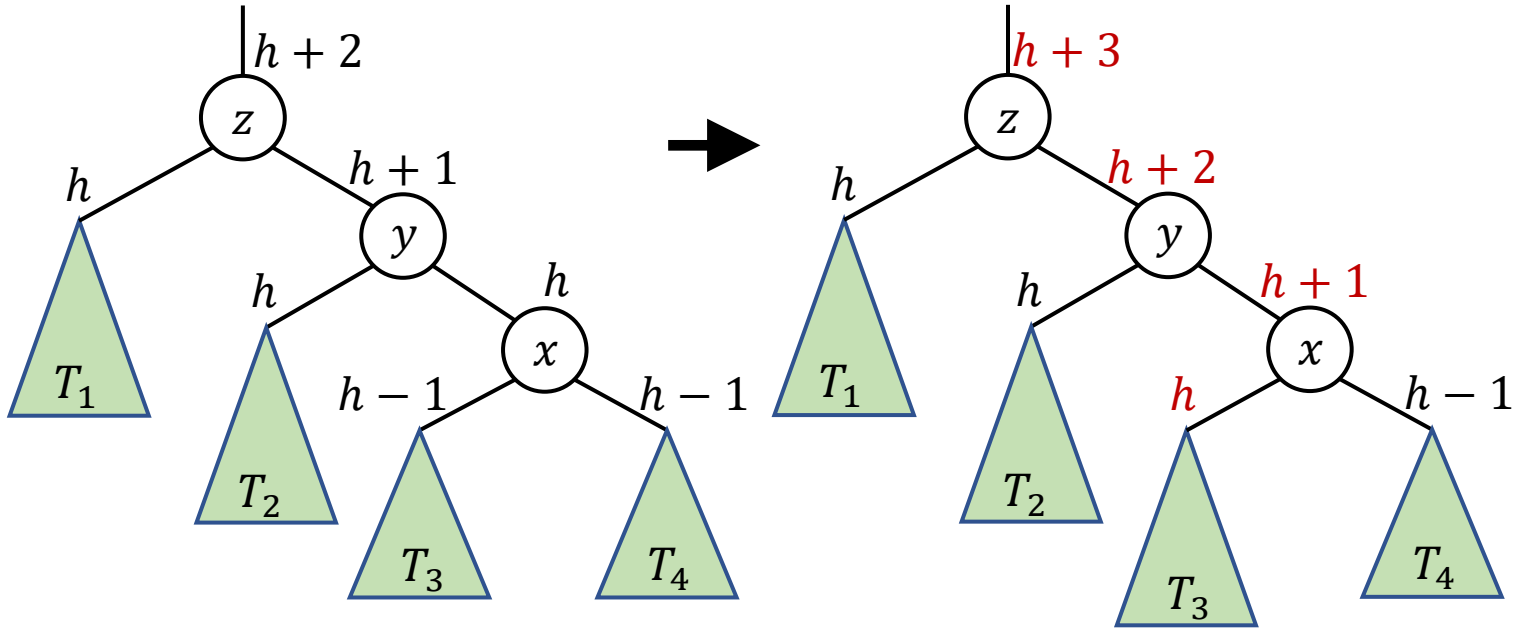
if z does not exist then return -1

else return z.height



Case 2 (RR Rotation)

- Insertion into **the right subtree of the right child** of node z



Case 2 (RR Rotation)

RightRight(z):

y ← z.right

z.right ← y.left

y.left ← z

z.height ← max(height(z.left), height(z.right)) + 1

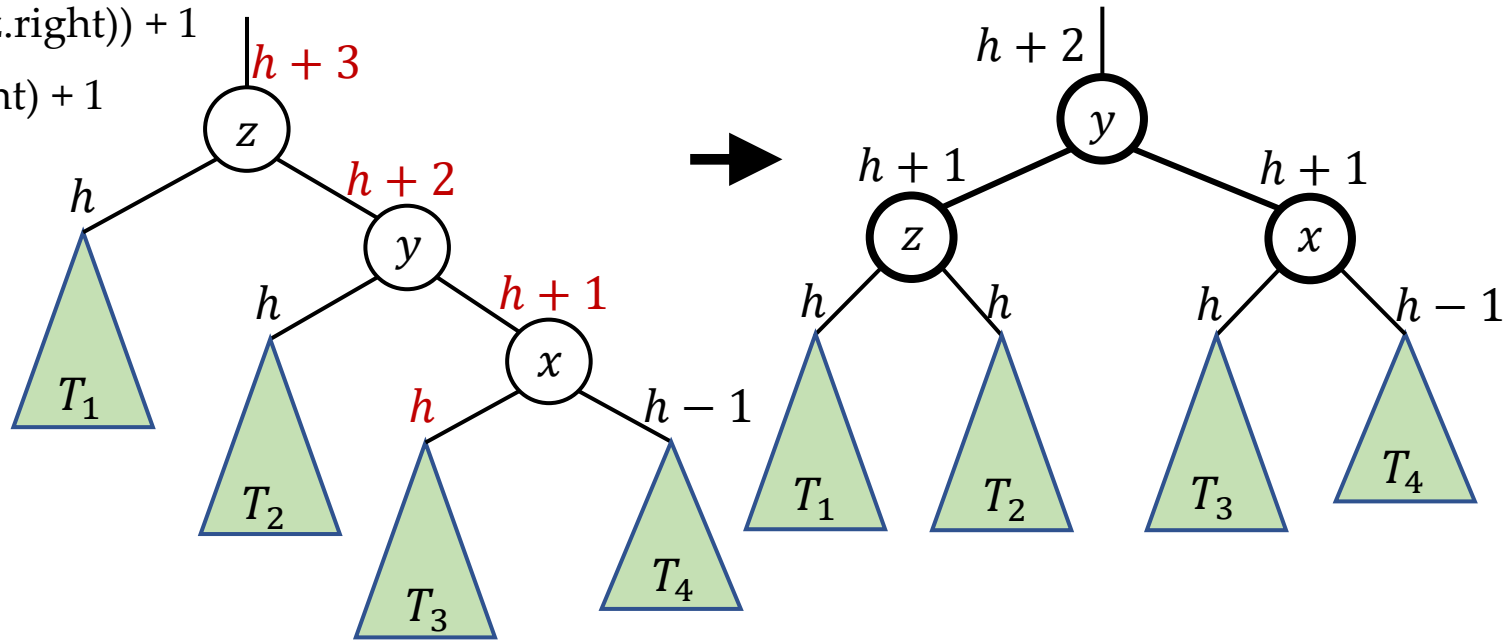
y.height ← max(height(y.right), z.height) + 1

return y /* new root */

height(z):

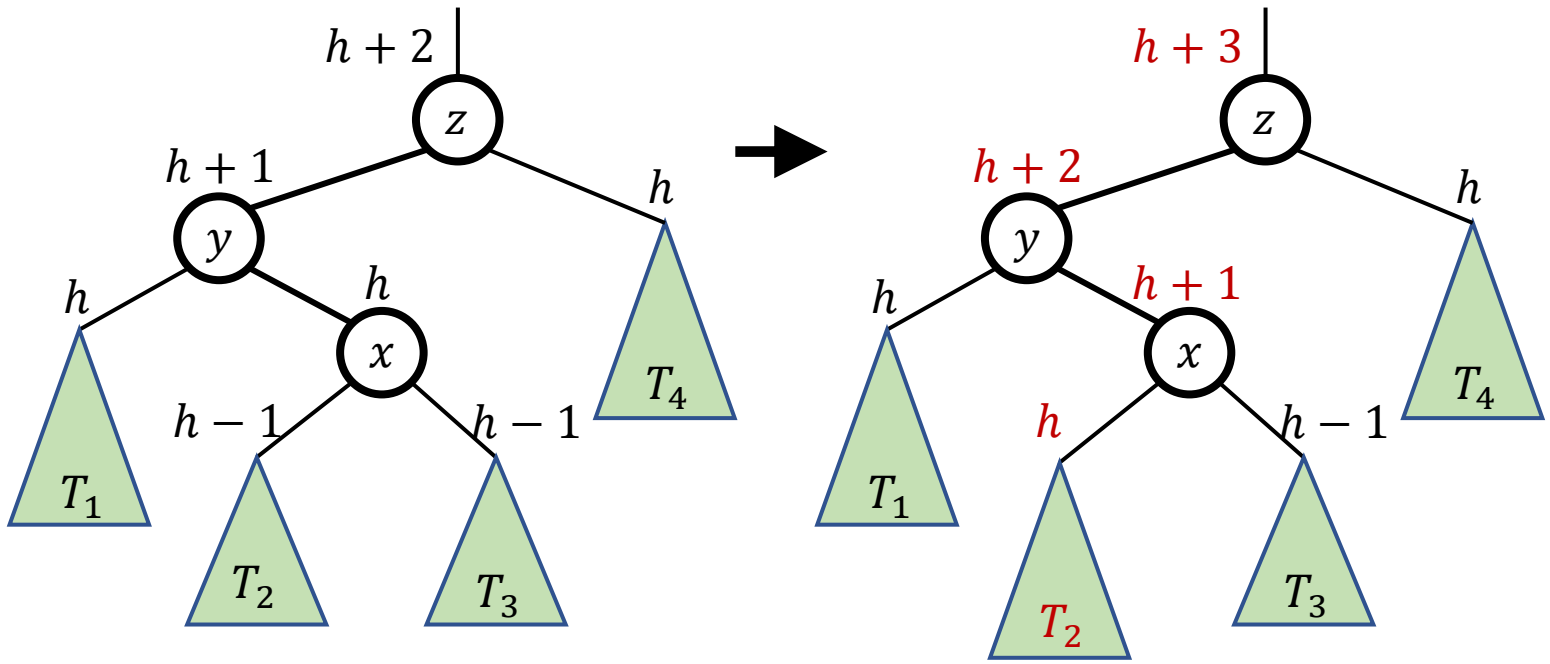
if z does not exist then return -1

else return z.height



Case 3 (LR Rotation)

- Insertion into **the right subtree of the left child** of node z

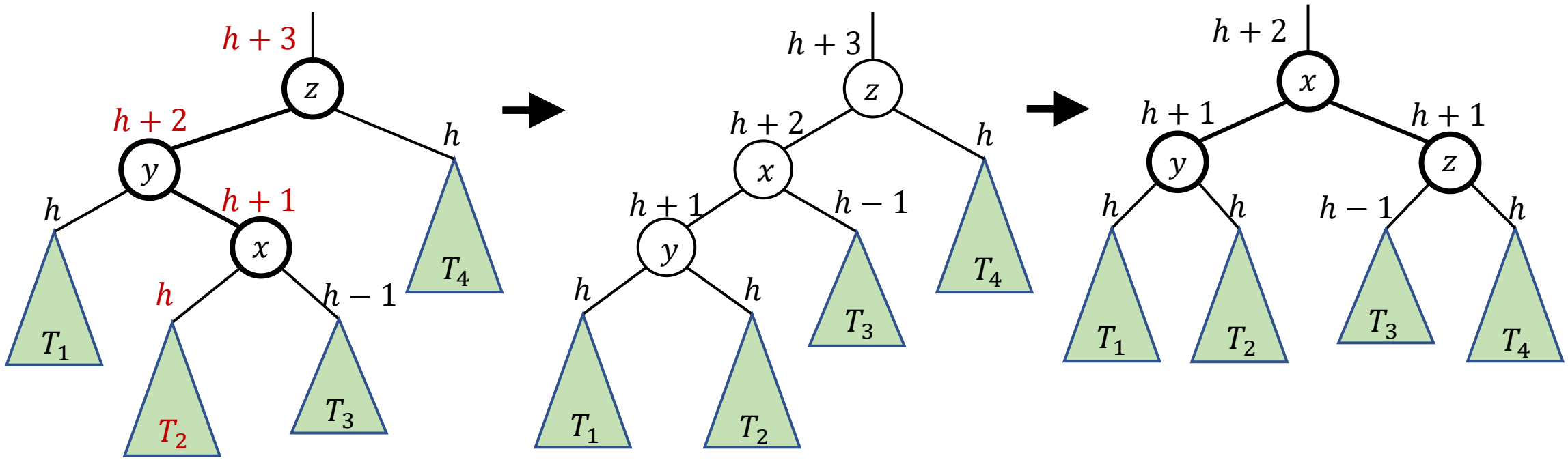


Case 3 (LR Rotation)

LeftRight(z):

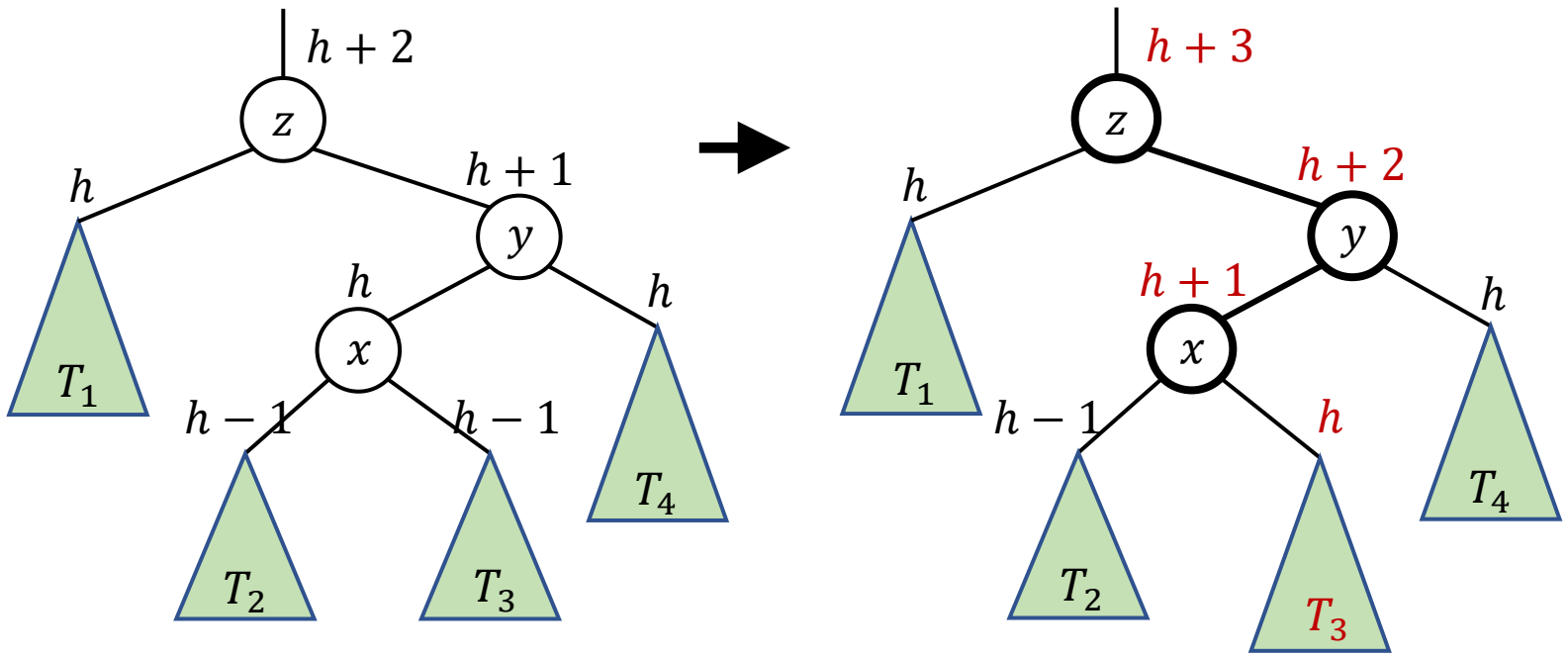
z.left = RightRight(z.left)

return LeftLeft(z)



Case 4 (RL Rotation)

- Insertion into **the left subtree of the right child** of node z

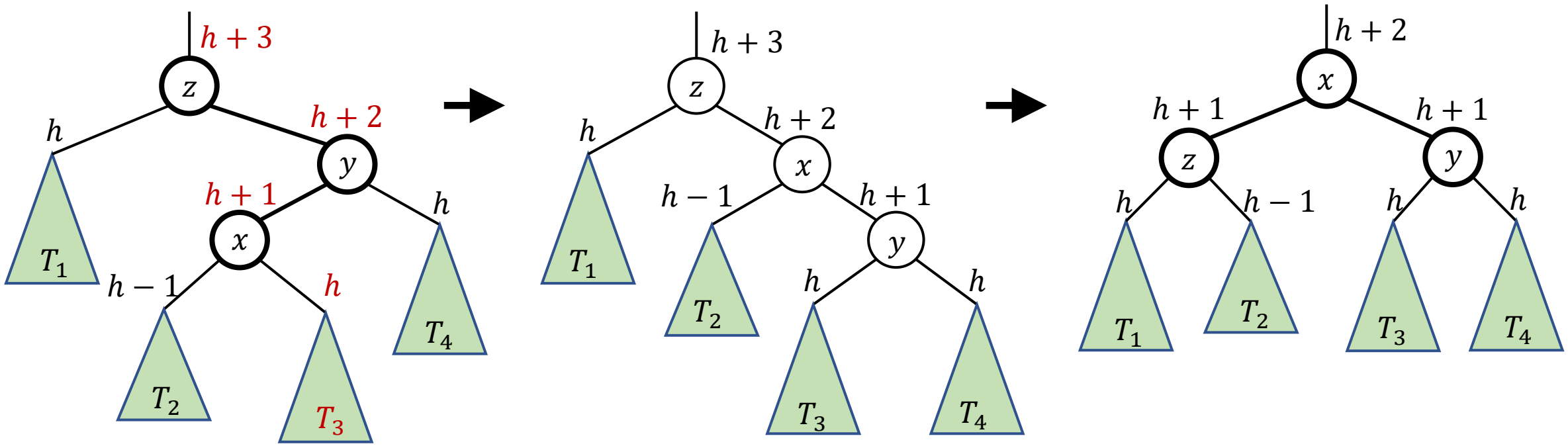


Case 4 (RL Rotation)

RightLeft(z):

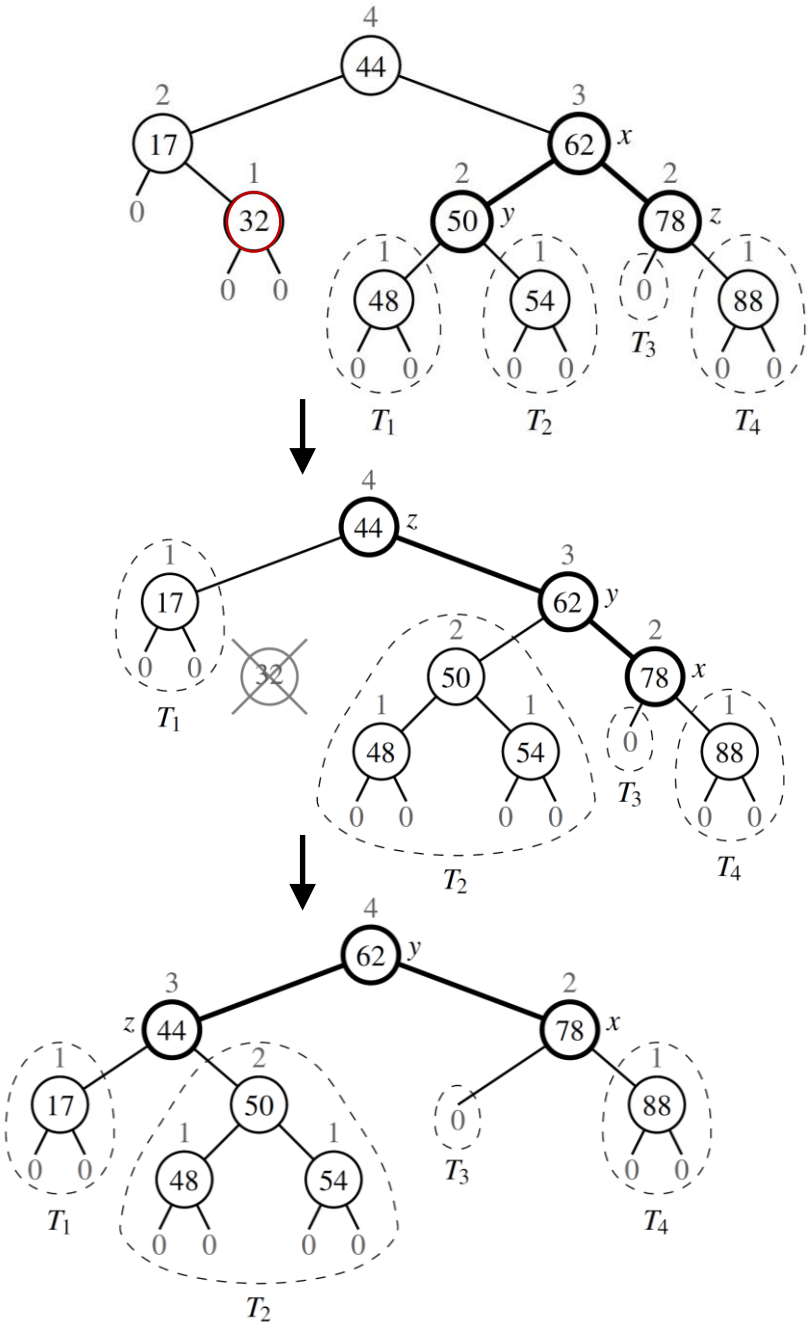
z.right = LeftLeft(z.right)

return RightRight(z)



Deletion

- Suppose that we delete a child of node p.
  - There can be an unbalanced node on the path from p to the root.
- z: the nearest ancestor of p that became unbalanced after the deletion
- y: the child of z with larger height
- x:
  - the taller child of y if one of the children of y is taller than the other
  - the child of y on the same side as y otherwise
    (if y is the left child of z, let x be the left child of y,
    else let x be the right child of y)
- Perform **Restructure(x)**.



```
Delete(root, key)
  if no root exists
    return root
  else if key < root.key
    root.left ← Delete(root.left, key)
  else if key > root.key
    root.right ← Delete(root.right, key)
  else   // if key == root.key
    if no root.left exists
      Release root and return root.right
    else if no root.right exists
      Release root and return root.left
    succ ← Get a successor node with min in root.right
    root.key ← succ.key
    root.right ← Delete(root.right, succ.key)
  if no root exists
    return root
```

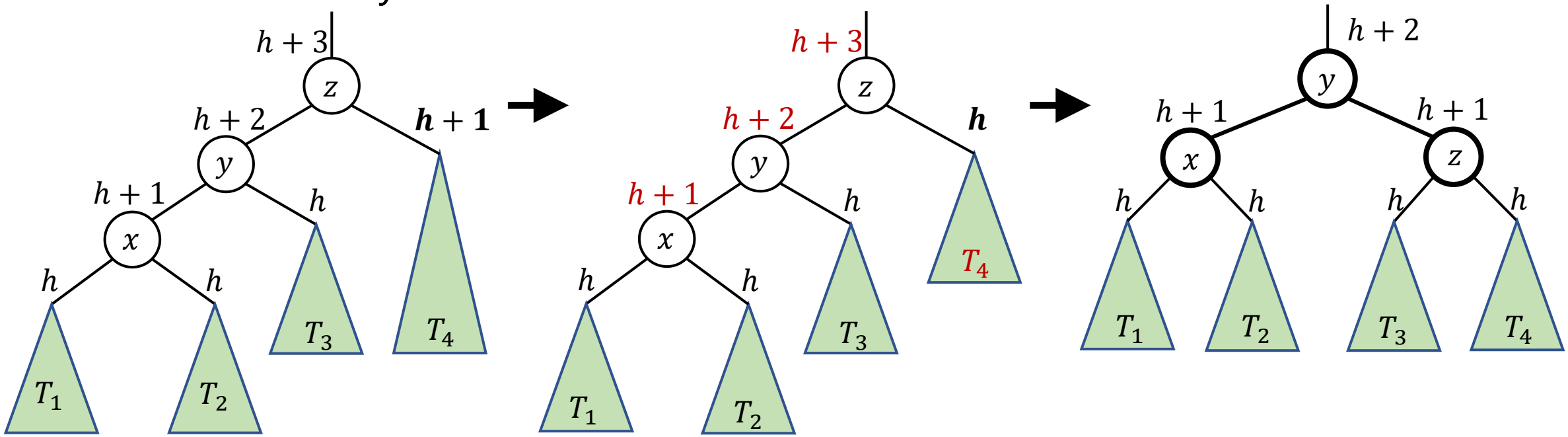
- **Update the height of the root**

```
  root.height ← 1 + max(height(root.left), height(root.right))
  if height(root.left) – height(root.right) > 1
    if height(root.left.left) – height(root.left.right) >= 0
      root ← LeftLeft(root)
    else
      root ← LeftRight(root)
  if height(root.right) – height(root.left) > 1
    if height(root.right.left) – height(root.right.right) >= 0
      root ← RightLeft(root)
    else
      root ← RightRight(root)
  return root
```

- Balance the root of the current subtree if it becomes unbalanced

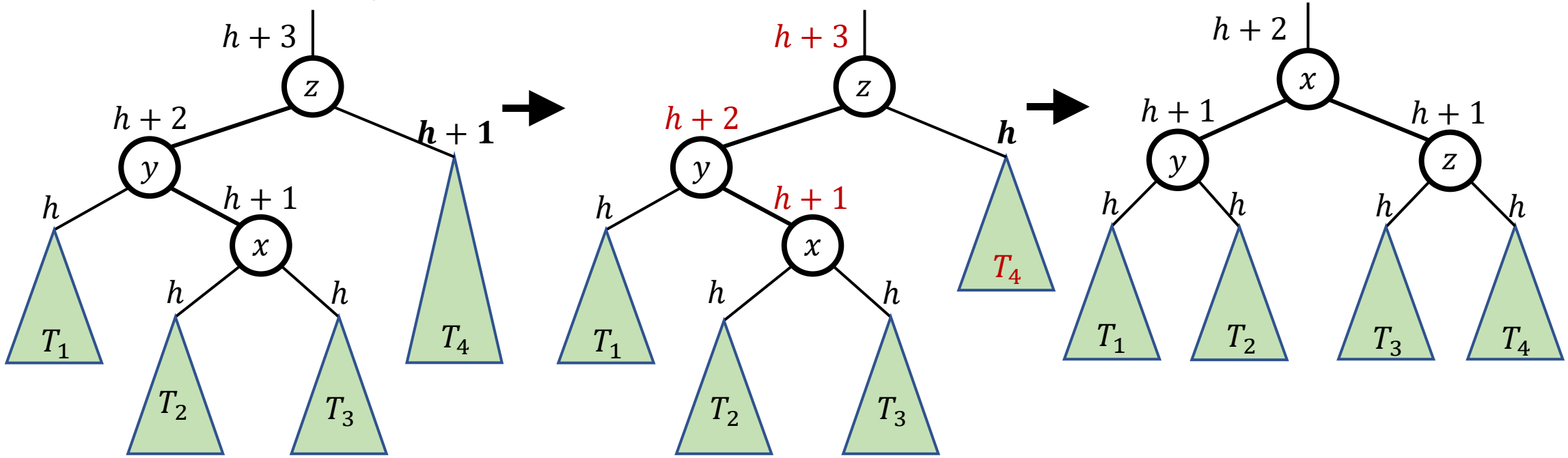
Case 1 (LL Rotation)

- Deletion of a node from T_4 decreases the height of T_4
  = Insertion into **the left subtree of the left child** of node z
- RR rotation is symmetric to LL rotation

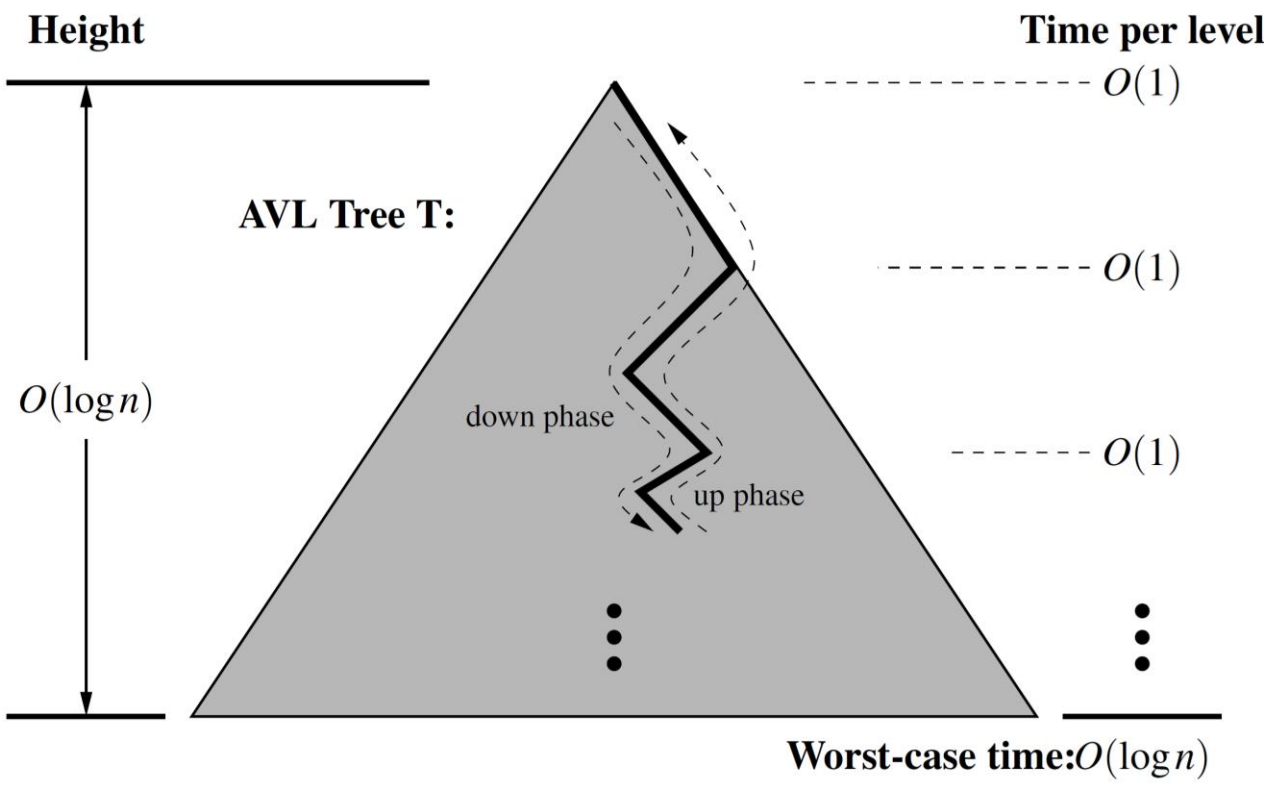


Case 3 (LR Rotation)

- Deletion of a node from T_4 decreases the height of T_4
  = Insertion into **the right subtree of the left child** of node z
- RL rotation is symmetric to LR rotation



Time Complexity



Operation	Running Time
Search	$O(logn)$
Insertion	$O(logn)$
Deletion	$O(logn)$

- Time complexity is $O(1)$ per level, broken into a down phase and an up phase.
- Down phase: searching
- Up phase: updating heights, performing trinode restructurings